Jinx: Explainable Duplicate Warnings for Pull Requests from Coding Agents

Harshul Jain
Independent Researcher

Saurabh Yergattikar
Independent Researcher

Tanya Sah
Independent Researcher

Parv Khatri
Independent Researcher

Abstract

Coding agents open many pull requests (PRs), and some of them repeat work that is already open in the same repository. Jinx is a GitHub Action and command-line tool that compares each new PR with the repository's open PRs and posts one comment listing likely duplicates, with a plain-language reason for each match: same files, identical added lines, same issue, similar description. It runs on the CI runner's CPU, needs no API key and sends no code to an outside service. Maintainers can tune one threshold to trade missed duplicates against false alarms. On 313 historical cross-author duplicate pairs from the AIDev dataset, each scored against the PRs that were open at the time, Jinx ranks the true duplicate first 0.917 of the time in its default mode, statistically tied with a classic research detector and a simple link rule, and raises 1.9% false alarms, against 5.5% for the best existing open-source duplicate-PR bot.

CCS Concepts

• **Software and its engineering → Software evolution**.

Keywords

duplicate pull requests, coding agents, GitHub Actions, code review

1 Introduction

AI coding agents now open hundreds of thousands of PRs on GitHub [5], and duplicated work is a leading reason why they are not merged [1, 2]. Duplicates appear when several agents, or an agent and a person, work on the same issue, or when an agent opens a new PR instead of updating its old one. Each duplicate costs a maintainer a review that did not need to happen.

Existing options leave a gap. GitHub lets maintainers mark duplicates only by hand. Small open-source bots (contrib-guard, Simili, PRSense, PRism, vector-triage, doppelgänger[1]) mostly compare titles and descriptions, and some need a paid LLM API. Research detectors [4, 6–9] are not packaged as maintained tools.

Jinx is built around what a maintainer needs from a warning: it should be right when it speaks, explain itself so it can be dismissed quickly, and run inside the repository's own CI. A companion study [3] found that, on open-only queues, several methods rank duplicates about equally well but differ widely in false alarms, and that duplicates often go unnoticed for more than a day. Jinx therefore aims for a packaged, explainable detector with competitive false alarms rather than for a new ranking algorithm.

[1] https://github.com/dannyl1u/doppelganger

2 How Jinx Works

Fig. 1 shows the five steps.

1. Read PRs. When a PR is opened, the Action reads the new PR and up to the 200 most recent open PRs through the GitHub API with the workflow's own token: title, description, linked issues, changed files and code changes. The command-line tool reads the same fields from JSON files.

2. Clean text. Agents fill PR descriptions with boilerplate such as checklists and "Generated with …" footers, which makes unrelated PRs look alike. Jinx learns which lines repeat across a repository's PRs and removes them.

3. Compare each pair. Text signals measure how similar the titles and descriptions are (TF-IDF and, optionally, a MiniLM embedding that runs locally). Code signals measure shared changed files, overlapping changed line ranges and identical added lines. Context signals record a shared linked issue, a mention of the candidate's PR number in the new PR, the same author or agent, and the time between the PRs.

4. Score and explain. A logistic regression model, trained on the AIDev benchmark with open-only queues and shipped with the package, turns the signals into a score between 0 and 1. Jinx flags a candidate at 0.5, where in the benchmark it flags about half of the cross-author duplicates; maintainers can raise or lower this threshold. Every signal that pushes the score up becomes one short sentence.

5. Report. The command-line tool prints a ranked list (Fig. 2). The Action posts one comment on the new PR and edits that comment when the PR is updated. Jinx never closes, labels or blocks a PR.

3 Using Jinx

A repository adds this workflow:

```
on:
  pull_request_target:
    types: [opened, reopened, synchronize, edited]
permissions:
  contents: read
  pull-requests: write
  issues: write
jobs:
  jinx:
    runs-on: ubuntu-latest
    steps:
      - uses: ABCR-Labs/Jinx@v1
        with:
          embed: "false"   # "true" adds MiniLM
```

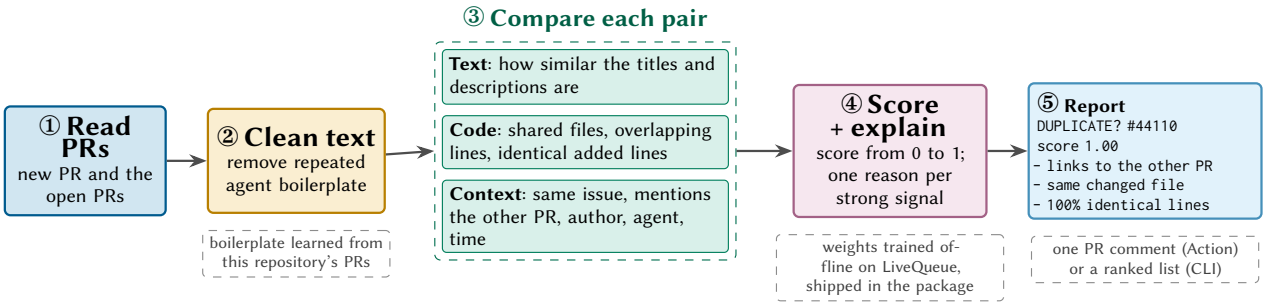


Figure 1: How Jinx works. It compares a new PR with each open PR using text, code and context signals, combines them into one score, and explains each likely match in plain words.

```
$ jinx check new.json --open-queue open.jsonl
PR #44134: checked 19 open PRs in 12 ms.
1 likely duplicate(s) (score >= 0.50).
  DUPLICATE? #44110  score 1.00  refactor(tvp):
     use capture_outputs for output tracing
    - one PR links to the other (#44110)
    - shares 1 of 1 changed files
      (src/transformers/models/tvp/modeling_tvp.py)
    - 100% of changed line ranges overlap
    - 100% of added lines are identical
    - similar title/description after removing
      agent template text (0.92)
  #44133  score 0.05  refactor(poolformer): ...
    - similar title/description (0.50)
    - same author (possible replacement PR)
```

Figure 2: Jinx on a real agent PR from huggingface/transformers, taken from AIDev (lexical mode). The second candidate is listed but not flagged.

Security. The workflow uses `pull_request_target` so that it can comment on PRs from forks. This trigger runs with write permissions, so Jinx never checks out or runs the PR's code: it reads metadata and diffs through the API only. *Speed.* Scoring takes 6 ms per PR at the median in the default lexical mode and 0.4 s with MiniLM (4 CPU threads, model already loaded). These times exclude installing the package, fetching PRs through the API (one call per PR for its files) and, with MiniLM, installing PyTorch and downloading the 90 MB model on an uncached runner, which is why MiniLM is off by default. MiniLM does not change Recall@1 significantly (Table 1).

Limits. The Action looks only at the 200 most recent open PRs; in the benchmark this hid the true duplicate in 11 cross-author cases. Locally, the `jinx check` and `jinx learn-templates` commands read PRs from JSON files.

4 Evaluation

We used the AIDev dataset [5]: 1767 pairs of agent PRs that a comment marks as duplicate or superseded, each scored against the PRs that were open in the repository when the newer PR was created, plus 2965 PRs with no duplicate. Weights are trained with five repository-grouped folds for evaluation; the shipped weights are retrained on all data with the same recipe.

Table 1: Cross-author duplicate pairs on open-only queues (n = 313, historical AIDev data [3]). Flagged: true duplicate ranked first and above the method's threshold. False alarms on 2965 PRs without a duplicate, 95% CIs.

Method	R@1	Flagged	False alarms %	Explains code
Jinx, default (lexical)	0.917	47%	1.9 [1.3, 2.7]	yes
Jinx with MiniLM	0.927	46%	1.8 [1.2, 2.5]	yes
INTRUDE + time	0.914	47%	2.0 [1.3, 3.0]	no
contrib-guard (bot)	0.914	46%	5.5 [4.0, 7.0]	no
Link rule	0.936	47%	1.2 [0.7, 1.7]	no

Table 1 compares the shipped modes with a classic detector (INTRUDE [8] with a time feature [9]), the best existing bot and a link rule. Jinx's ranking is statistically tied with the classic detector and the link rule; we do not claim better ranking. Its false alarms are below the best bot's (difference +3.7 points, CI +2.2 to +5.1) and close to INTRUDE + time and the link rule. What Jinx adds over these is packaging and explanation: unlike the link rule and the bot, it explains code-level matches (shared files, identical lines), and unlike the research detector it ships as an installable Action. On the 118 cross-author pairs with no link at all, no method is clearly better (Jinx 0.839).

Reading a warning. Fig. 2 shows a true duplicate, where five reasons agree. Fig. 3 shows a warning that is wrong: a follow-up PR stacked on the same author's earlier PR. Because each reason is listed, a maintainer can see what triggered the warning without opening both diffs. Stacked PRs are a known source of false alarms for any code-overlap signal. Fig. 4 shows the Action's comment from a live run on the public Jinx repository, where we opened four demonstration PRs, three of them for the same issue. Jinx commented on the PR that redid an open PR's change (score 1.00) and stayed quiet on an unrelated documentation PR. A third PR for the same issue, written with different wording and a different implementation, scored 0.30 and got no comment at the default threshold; that is the kind of duplicate a lower threshold would catch, at the cost of more false alarms. These were our own test PRs, not a maintainer deployment.

```
$ jinx check new.json --open-queue open.jsonl
1 likely duplicate(s) (score >= 0.50).
  DUPLICATE? #901  score 0.59  Add GitHub pull
     request opened trigger
    - shares 5 of 10 changed files (...)
    - 100% of changed line ranges overlap
    - 100% of added lines are identical
    - similar title/description (0.60)
    - same author (possible re-opened or
      replacement PR)
```

Figure 3: A real false alarm from the benchmark (giselles-ai/giselle #904, "Add GitHub pull request closed event", labelled as having no duplicate). The new PR appears to build on the same author's earlier PR, so it repeats that PR's code. The reasons show why Jinx flagged it; the same author and a different title suggest a follow-up PR rather than a duplicate.

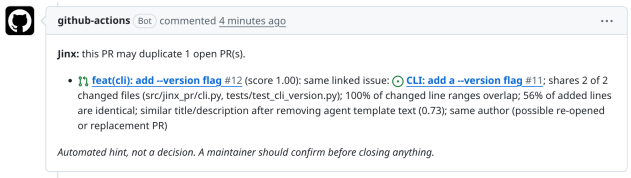


Figure 4: The Jinx Action's comment on a live demonstration PR (ABCR-Labs/Jinx #15), which redoes open PR #12 for the same issue #11.

5 Limitations

Jinx often misses duplicates that change different files to reach the same goal (0.720 Recall@1 on such cross-author pairs). Labels come from PR comments; an LLM audit judged 84.3% of the cross-author labels to be true duplicates or replacements, and a human check is pending. Apart from the demonstration run, we have evaluated Jinx on historical data only; no maintainer has used it yet, and a run on live repositories is future work. In the companion study, asking Claude Haiku 4.5 to check Jinx's top five candidates cut false alarms from 1.7% to 0.8% for about 0.8 US cents per PR [3]; Jinx does not offer this optional check yet.

6 Availability

Jinx is open source under the MIT licence at https://github.com/ABCR-Labs/Jinx and installs with `pip install` git+https://github.com/ABCR-Labs/Jinx; the Action is used as `ABCR-Labs/Jinx@main`. A PyPI release (`jinx-pr`) and a short demo video will follow before submission.

References

[1] Khairul Alam, Saikat Mondal, and Banani Roy. 2026. Why Are AI Agent Involved Pull Requests (Fix-Related) Remain Unmerged? arXiv:2602.00164.
[2] R. Ehsani et al. 2026. Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub. MSR 2026.
[3] Harshul Jain, Saurabh Yergattikar, Tanya Sah, and Parv Khatri. 2026. How You Build the Queue Decides the Winner: Re-Evaluating Duplicate Pull Request Detection on Coding-Agent PRs. Under review.
[4] Jing Jiang, Qiudi Li, Jin Guo, Zhenrui Li, Li Zhang, and Ge Li. 2023. DupHunter: Detecting Duplicate Pull Requests in Fork-Based Development. *IEEE Trans. Softw. Eng.* (2023). doi:10.1109/TSE.2023.3235942
[5] Hao Li et al. 2026. AIDev: Studying AI Coding Agents on GitHub.
[6] Zhixing Li, Gang Yin, Yue Yu, Tao Wang, and Huaimin Wang. 2017. Detecting Duplicate Pull-Requests in GitHub. In *Proc. Internetware.* doi:10.1145/3131704.3131725
[7] Zhifang Liao, Yuanyuan Liu, Yuqi Lan, and Yan Sun. 2024. DupLLM: Duplicate Pull Requests Detection Based on Large Language Model. In *Proc. APSEC.* doi:10.1109/APSEC65559.2024.00023
[8] Luyao Ren, Shurui Zhou, Christian Kästner, and Andrzej Wąsowski. 2019. Identifying Redundancies in Fork-based Development. In *Proc. SANER.* doi:10.1109/SANER.2019.8668023
[9] Qingye Wang, Bowen Xu, Xin Xia, Ting Wang, and Shanping Li. 2019. Duplicate Pull Request Detection: When Time Matters. In *Proc. Internetware.* doi:10.1145/3361242.3361254